

M U A I

System Ownership Audit

*A complete picture of what Mu AI is, what it's made of,
what runs where, what costs money, and what you own.*

1. Executive summary

Mu AI is a multi-tenant AI sales bot for Instagram DMs. Right now it serves one client (Coach Shaun, Fairway Performance Golf). The architecture, database, and dashboard are built to serve many clients; the only single-client piece left is the Worker configuration, which is a small change when the second coach onboards.

The system is live, processing real conversations daily, and the bot is generating replies that setters review and send. The infrastructure runs on your accounts (Cloudflare, Supabase, Make, Anthropic), so the operating cost of the platform is yours and so is the data. The code lives in a GitHub repository under my account, public.

We are currently in Phase 2: getting Meta App Review approved so the system can be white-label resold to other coaches, and so we can pull inbound conversation history directly from Meta instead of relying on ManyChat as middleware.

2. What you own

Plain language. If something goes wrong with me tomorrow, here's what's yours and stays yours.

The product

- Mu AI as a brand and software platform
- The bot's accumulated learnings (every setter correction, every conversation pattern Claude has been trained against)
- All conversation history, lead data, and analytics in your Supabase database

The infrastructure (under your accounts)

- Cloudflare account: hosts the Worker (the brain) and the dashboard (the inbox UI)
- Supabase account: holds the database (conversations, reviews, learnings, audit logs)
- Anthropic account: pays for the AI behind the replies
- Make.com account: orchestrates the inbound and outbound message flow

The domain

- getmu.co, registered at GoDaddy under your account

The code

- Source code lives at github.com/iamanthony1007/botos-platform, currently under my GitHub account
- The repository is public, so you can read it anytime

- We should plan a transfer to a GitHub organization you own as part of the white-label handover, but it's not blocking anything right now

What is NOT yours (so you don't get surprised)

- ManyChat is on Coach Shaun's account with his subscription. When you onboard other coaches, each coach will have their own ManyChat workspace on their own account.
 - My GitHub account holds the repo today. Transferring it to you is a one-click action when you're ready.
 - The **Anthony** name on git commits is mine. That's just the author history of the code; it doesn't affect ownership of the codebase itself.
-

3. How it works

Here's what happens when a lead messages Coach Shaun on Instagram.

1. Lead types a message on Instagram
|
2. ManyChat receives it (ManyChat is connected to Meta)
|
3. ManyChat fires a webhook to Make.com (Scenario 1)
|
4. Make formats the data and sends it to our Cloudflare Worker
|
5. The Worker:
 - Looks up the lead in your Supabase database
 - Loads conversation history and context
 - Sends everything to Anthropic's Claude API
|
6. Claude generates a reply (text plus metadata like sales stage and intent)
|
7. The Worker decides:
 - Either auto-send (high confidence, early stage) -> straight to step 8
 - Or queue for setter review -> setter sees it in the dashboard
|
8. Worker sends the reply to Make.com (Scenario 2)
|
9. Make.com sends it to ManyChat
|
10. ManyChat delivers it to the lead on Instagram

End to end, this takes 2 to 5 seconds. The full conversation is stored in Supabase as it happens, so the dashboard always shows the live state.

Why ManyChat is in the picture: Meta requires App Review for direct API access to send Instagram DMs at scale. We don't have App Review yet (that's Phase 2). Until we do, ManyChat acts as the bridge between Meta and our system. Once App Review is approved, ManyChat becomes optional and we can talk directly to Meta's API.

4. Services and tools (with costs)

Every external service the platform depends on. For each one, what it does, whose account it's under, and what it costs.

Infrastructure (third-party platforms)

Service	What it does	Account holder	Monthly cost
Cloudflare Workers	Runs the Worker (the brain)	Nella	\$0 (Free plan; cron triggers are included free, current volume well under the 100,000 daily request cap)
Cloudflare Pages	Hosts the dashboard (the inbox UI)	Nella	\$0 (Free plan, plenty of headroom)
Cloudflare KV	Fast memory cache for conversations	Nella	\$0 (Free plan tier sufficient)
Supabase Pro	Database (conversations, reviews, learnings, audit log)	Nella	\$25
Anthropic (Claude API)	The AI that generates replies	Nella	~\$100 currently. This is the biggest lever for cost reduction. See "Cost reduction plan" below.
Make.com Core (40,000 credits/month)	Orchestrates the message flow	Nella	\$34.12 (currently using ~21% of monthly credits, comfortable headroom)
ManyChat	Bridges to Instagram for now	Coach Shaun	His subscription. Will be replaced per-client once Meta App Review approves and we move to direct API.

Service	What it does	Account holder	Monthly cost
GoDaddy (domain getmu.co)	Domain registration	Nella	~\$2.50 (about \$30/year amortized)
GitHub	Source code repository	Anthony (currently, to be transferred)	\$0 (public repo)

Infrastructure subtotal: approximately \$162/month, paid directly to the platforms above.

Platform management

Service	What it covers	Paid to	Monthly cost
Mu AI platform management	Ongoing maintenance, bug fixes, monitoring, deploys, small feature updates, setter support questions, scenario tweaks in Make, schema changes in Supabase	Anthony	\$400

Management fee may increase as the platform scales. Adding new coaches, higher message volume, additional automations, or new integrations beyond the current scope can change the work envelope. Any increase would be agreed in writing before it takes effect, never billed by surprise.

Total monthly cost

Approximately \$562/month at current Coach Shaun-only volume. Infrastructure \$162 + management \$400. Coach Shaun's ManyChat sits separately on his card. The Anthropic line (\$100) is the biggest infrastructure lever and is addressed below.

Cost reduction plan: Anthropic API

You flagged the \$100/month Anthropic spend as too high and asked for ways to bring it down significantly. There are real levers here.

Lever 1: Prompt caching (biggest impact, already partially staged)

Every time the bot replies, the Worker sends Claude the entire system prompt (the bot's personality, sales psychology, product knowledge, learnings), the conversation history, and the new message. The system prompt is the same on every call, but right now we pay full input-token rate for it every time.

Anthropic offers prompt caching, where stable parts of the prompt are cached server-side and charged at roughly 10% of the normal rate. For a system like ours with a long stable prompt and short variable messages, this typically cuts Anthropic costs by 50 to 70%.

Status: a `feat-prompt-caching` branch already exists in the repo. It was deferred earlier this year pending a separate schema fix. Re-prioritizing this is the single biggest win available.

Realistic outcome: \$100/month -> roughly \$30 to \$50/month with prompt caching active.

Lever 2: Model tier review

We currently use Claude Sonnet for every reply. For simple turns (greeting responses, simple acknowledgements, follow-up nudges), Claude Haiku at roughly one-fifth the cost would work fine. A two-tier model approach (Haiku for simple turns, Sonnet for complex sales moments) is feasible but adds engineering complexity.

Realistic outcome: another 10 to 20% reduction on top of prompt caching, but worth doing only if Lever 1 doesn't get us low enough.

Lever 3: Reduce memory window

The Worker currently includes the last 15 messages of conversation in every call. For most leads, 8 to 10 is enough. Reducing this would cut input tokens proportionally. Tradeoff: bot might forget context further back in long conversations.

Realistic outcome: 10 to 15% reduction. Lower risk if we cap reductions carefully.

Lever 4: Auto-send threshold tuning

When the bot auto-sends, the call still costs the same as when it queues for review. But we currently regenerate replies on certain re-engagement events. There's a known bug (Lead-source-event re-fires, already in the work queue) that causes wasteful regeneration. Fixing that saves both tokens and setter confusion.

Realistic outcome: small but real (~5%).

Combined target after all four: \$100/month -> \$25 to \$40/month range, depending on lead volume changes.

Recommendation: Prioritize Lever 1 (prompt caching) as the next engineering workstream after Phase 2 (Meta App Review). I'll scope it as a separate piece of work with its own price quote when we're ready.

5. Features currently live

What the system can actually do today, in product terms.

The AI sales bot

- Reads every inbound Instagram DM (via ManyChat as the current bridge)
- Generates personalized replies in Coach Shaun's voice, based on his configured system prompt, sales psychology, and product knowledge
- Tracks where each lead is in the sales journey (a "stage": cold, warming, qualified, scheduled, booked, etc.)
- Detects intent (high, medium, low based on lead's signals)
- Auto-sends replies when confidence is high and stage is early; otherwise queues for setter review

The setter inbox (dashboard)

- Real-time list of leads needing a response
- Full conversation thread view
- Setter can approve the AI's suggested reply as-is, edit and send, or discard and write their own
- Every edit becomes a "learning" that improves future replies

Lead management

- Filter leads by stage, intent, status
- "Closest to Booking" view that surfaces highest-priority leads
- Search by lead name or Instagram username
- Manual escalation handoff phrases (so the bot stops auto-replying when a human takes over)

Auto follow-up cron

- Every hour the system scans for leads who messaged 20-21 hours ago and haven't been followed up
- Sends a single low-touch follow-up (`<firstname>?`) before the Instagram 24-hour messaging window closes
- Skips leads with no proper name set, leads currently being handled by a human, leads at booked stage, and testers
- Capped at 50 sends per hour for safety
- Live in production as of 2026-05-12; fix to the message delivery shape applied 2026-05-18

Learnings system

- Every setter correction is stored as a "learning"
- Future AI replies see these learnings as context, so the bot learns from human corrections

- Settings page lets you view, edit, or delete learnings

Analytics dashboard

- Conversion funnel (cold to booked)
- Intent distribution
- Stage breakdowns
- Per-day message volume

Audit log

- Every identity edit (setter changing a lead's name, intent, stage manually) is recorded with timestamp and user
- Useful for understanding why a lead's record looks the way it does

Bot tester

- A Conversation Simulator on the Settings page where you can test how the bot replies to a synthetic message before changing the live prompt
- Useful for prompt tuning without affecting real leads

6. What we're currently working on

Phase 2: Meta App Review and Inbound Visibility.

Why this matters for you

- Right now Mu AI depends on ManyChat to send and receive Instagram messages. That works but it's a middleware dependency, and ManyChat costs money per-client.
- Without Meta App Review, you can't legally white-label resell this system to other coaches.
- Without Inbound Visibility (Phase 1 of post-approval work), the platform misses replies that coaches type manually in the Instagram app or in ManyChat's Live Chat. Those don't currently flow through our pipeline, so the setter inbox shows an incomplete picture.

What we're doing

- Pre-flight checks with you (see the project plan I sent you in the WhatsApp message, separate document)
- Domain DNS swap so getmu.co points at Cloudflare and serves a basic landing page
- Drafting privacy policy, hosted at getmu.co/privacy
- Building the Meta-required deauthorize and data deletion endpoints on the Worker
- Registering the Meta App, requesting three permissions

- Recording screencasts that demonstrate each permission in use
- Submitting App Review and responding to any reviewer feedback
- After approval: building the Inbound Visibility integration so manual replies show in the dashboard

Timeline 4 to 8 weeks wall clock, mostly Meta's review queue. Most of the work hours are mine; what you need to provide is in the project plan document.

Cost Already covered separately in the project plan. \$1200 total, payable 50/50 (half upfront, half on Meta approval).

7. Known limitations

I'm being honest about the gaps so you have a complete picture. None of these are crises, but each is real.

Single-tenant Worker, multi-tenant database The database is set up to handle many coaches, but the Worker right now has Coach Shaun's bot ID hardcoded. When you onboard a second coach, we need to either deploy a separate Worker per client or modify the Worker to figure out which client based on the incoming webhook. About 30 minutes of work, but worth knowing it's still pending.

Security boundaries are not yet production-grade

- The Worker's webhook endpoint is currently public (no authentication on incoming requests). For now, this is fine because the URL isn't widely known and the endpoint validates the data it receives. Long term we should add a shared secret or signed requests. This is on my list.
- The dashboard uses Supabase's row-level security for access control, but the current policies are permissive. Any logged-in dashboard user can technically see any lead in the database. For now this is fine because the only logged-in users are you, me, and Coach Shaun's setters, all of whom should see everything anyway. When you onboard another coach with their own setters, we need stricter policies so coaches can't see each other's leads. On my list.

No automated alerting If something breaks at 3 AM, no one gets paged. The system writes errors to Cloudflare's logs but there's no monitoring dashboard, no Slack pings, no email alerts. We previously had a delivery-failure email alert wired through Resend; that's not currently active. Adding proper alerting (probably Sentry for errors, plus a simple Slack or Discord webhook for delivery failures) is a small project for after Phase 2.

No automated tests Code changes are tested manually by deploying to staging first and checking they work. No unit tests. This means we move slowly on changes and rely on staging to catch problems. Adding tests is a longer project for later.

Inbound visibility gap Replies typed manually in the Instagram app on phone, or replies typed in ManyChat's Live Chat, don't currently flow through our system. The dashboard inbox shows what the bot saw, not the full conversation. Phase 1 (after Meta App Review approves) fixes this.

Some setup details still hardcoded A few configuration values (Worker URLs in the dashboard, the production Make.com webhook URL) are written into the code rather than being environment variables. Fine for now, will need cleanup as we add more clients.

8. Access and credentials

What you have access to and what I have access to. This is an index of where things live, not the actual passwords.

What	Where to log in	Who has access
Cloudflare account (Workers, Pages, KV, DNS)	dash.cloudflare.com	You (account owner). I have collaborator access.
Supabase project (production)	supabase.com/dashboard	You (account owner). I have admin access via collaboration. The Worker also uses the service role key for backend writes.
Supabase project (staging)	supabase.com/dashboard	TBD - need to confirm whose account this is under
Anthropic console	console.anthropic.com	You (account owner). API key is set as a Worker secret.
Make.com (Scenarios 1 and 2)	eu2.make.com	You (account owner).
ManyChat	manychat.com	Coach Shaun
GoDaddy (domain)	godaddy.com	You
GitHub repository	github.com/iamanthony1007/botos-platform	Me (owner). Public repo, you can read without an account.
Production Cloudflare Worker URL	sales-bot.nellakuate.workers.dev	Public-facing endpoint, no login

What	Where to log in	Who has access
Production dashboard URL	botos-platform-3ar.pages.dev	Public URL, login required (Supabase auth)
Production database URL	rydkwsjwlgnivlwlvqku.supabase.co	Service role key in Worker, anon key in dashboard, your password for direct DB access

What I'd recommend you do

- Verify you can log in to each of those accounts independently of me
- Make sure the email address on each account is one you control long-term
- For Cloudflare and Supabase specifically, set up two-factor authentication if you haven't (these are the most consequential accounts)

9. Where the code lives

Two places. Both are important to know.

GitHub repository

- URL: `https://github.com/iamanthony1007/botos-platform`
- Status: public, anyone can read; only collaborators can write
- Branches: `main` is the production branch (always reflects what's live)
- Other active branch: `feat-meta-app-review-plan` (current work in progress)
- File structure:
 - `sales-bot/` - the Worker code (the brain)
 - `dashboard/` - the React dashboard code (the inbox)
 - `db/` - database schema and migrations
 - `docs/` - architecture and deployment docs
 - `PROGRESS.md` - single source of truth for project state
 - `SYSTEM-AUDIT.md` - technical audit (engineering reference)

What "deploying" means in practice

- When code changes on `main` and is pushed to GitHub, that's NOT automatically live. There's a manual deploy step.
- For the Worker: I run `wrangler deploy` from my local machine, which pushes the new code to Cloudflare.
- For the dashboard: I run `wrangler pages deploy dist --project-name=botos-platform`, which pushes the built static files to Cloudflare Pages.

- Staging is deployed first, then production. Always.
- Every deploy creates a version ID in Cloudflare. We can roll back to any previous version with one click if something breaks.

Why GitHub is currently under my account Just how I started it. It's a public repo so you can read every file without an account. When you set up a GitHub organization for your company (Mu AI, Inc. or whatever entity name you choose), I'll transfer the repository to that org. Transfer is one click for me, accept on your end. Existing collaborators, issues, and pull requests are preserved. URLs continue working via redirects.

Closing notes

This audit is current as of 19 May 2026. As we move through Phase 2 and beyond, sections 4, 5, and 6 will need updating. I'll keep this file in the repository so any future reference (or you, when you want to refresh your memory) can find it.

Questions or anything you want clarified, just ask.

Anthony